datafrag (runner patch) — guide

Revision: 1 **Last modified:** 2026-09-26T13:55:00Z

Source: `constitution/scripts/codegraph/runner_patches/datafrag1.py` (120 lines). A **module driven by `fk_index_patch.py`**, not a command. Citations are `datafrag1.py:<line>` unless another file is named. Machinery guide: fk_index_patch.

FINDING FIRST: this patch has no behavioural test

No file under `scripts/codegraph/tests/` contains `__helixDataFragClassify`, `data_fragment_skipped` or `binary_content_skipped`; the id appears only in selection arguments of other tests and in `codegraph_safe.sh` (lines 63-68, which state it has zero tests and is therefore not in the wrapper's default patch set). `owed_report.py` reports "tested" only because the id is named. This guide is traced from source plus the scratch checks under Usage. The patched `extraction/index.js` has **not** been run inside a real indexing pass.

Overview

Stock CodeGraph 1.6.0 hands three raw number-fragment C headers (comma-separated byte lists with no enclosing declaration) to tree-sitter, where each burns three 20 s hard-kill windows plus a retry, and it parses binary MPEG-TS `.ts` files as TypeScript. The single-file path used by `sync` (`indexFileWithContent`) parses on the main thread with no timeout at all (docstring lines 3-8).

datafrag1 classifies such files before parsing and stores them as **skip rows**: the file row exists with 0 nodes and one warning-severity error carrying a code, exactly like the vendor's `size_exceeded` path, so the file stays in the index (§11.4.78) and the vendor's `healZeroNodeRows` keeps it (lines 17-23).

Classification rule (lines 10-16, 35-76):

Class	Condition	Code
Data fragment	path ends in `.h .c .cc .cpp .hpp .inc .cxx .hxx`, size `>= 200 KiB` (204800), and the first character after leading whitespace and `/* */` / `//` comments, within the first 20,000 characters, is a digit `0-9`	`data_fragment_skipped`
Binary	decoded content contains a NUL character (`\u0000`), any extension	`binary_content_skipped`

The docstring records that numeric-ratio heuristics were rejected (869 candidates, 866 parsed fine) and that the rule matched exactly the 3 timing-out files across 584,604 (lines 12-15).

Prerequisites

Reads `lib/dist/extraction/index.js`. Python stdlib plus `runner_patches/common.py`. Default-set patch (`runner_patches/__init__.py:17`), applied third. The source must reference `cooperative_yield_1.createYielder` (line 112-113), used by the single-file branch.

Usage

Only through `fk_index_patch.py`:

```
python3 scripts/codegraph/fk_index_patch.py --patches datafrag1 --dst <scratch dir>
```

`NOT RUN` in this form (copies 282 MB). Run in a scratch directory holding only a copy of stock 1.6.0 `extraction/index.js`:

- `VERIFIED (ran: datafrag1.plan(FileView(scratch)); saw: APPLIED, anchors 'const MAX_FILE_SIZE', 'bulk loop: await feed(...)', 'indexFileWithContent: extractFromSource(...)'; detail extensions h|c|cc|cpp|hpp|inc|cxx|hxx, min_bytes 204800, window_chars 20000)`.
- `VERIFIED (ran: node --check on the planned file with system node v24.18.0; saw: exit 0; control: node --check on a broken file exits 1)`.
- `VERIFIED (ran: the injected __helixDataFragClassify extracted and run under node; saw: a .h file of 240000 bytes whose content starts "0x1,0x1,..." -> data_fragment_skipped; the same content 40 bytes -> null; a 210000-byte .h starting with "/* c */ // x int a;" -> null (declaration, not a digit); "ab\u0000cd" in m.ts -> binary_content_skipped; the digit fragment in a .py file -> null)`.

Behaviour (source-traced)

`plan(view)` (lines 106-120):

1. Refuse `source is ALREADY patched` if `__helixDataFragClassify` is present (lines 108-109).
2. NOT_NEEDED if the file already contains both `data_fragment_skipped` and `binary_content_skipped` (lines 110-111).
3. Refuse if `cooperative_yield_1.createYielder` is not referenced (lines 112-113).
4. Three `replace_once` edits (lines 115-117):
   - **constant anchor** `const MAX_FILE_SIZE = 1024 * 1024;` — replaced by itself plus the classifier helper;
   - **bulk loop anchor** `await feed(filePath, content, stats);` — classify first; on a hit call `storeResult` with 0 nodes, 0 edges, 0 refs and one `{severity: 'warning', code}` error, then `continue`; otherwise `feed` as before (lines 78-89);

- **single-file anchor** `const result = extractFromSource(...)` in `indexFileWithContent` — on a hit call `this.storeExtractionResult(...)` with the skip result and return it (lines 91-104).

The classifier (lines 40-75) returns `null` for non-string content, checks NUL first (any extension), then applies the extension and size gate, then scans up to 20,000 characters skipping whitespace and comments. If the scan runs out of the window inside a comment (unterminated `/*` or `//` within the window) or hits only whitespace, it returns `null` (not a fragment).

Edge cases

All `VERIFIED (ran: planning against a scratch copy of stock 1.6.0 extraction/index.js)`:

Condition	Result
Stock 1.6.0	APPLIED (3 anchors)
Plan again on the already-patched text	**Refuse**: `datafrag1: source is ALREADY patched — --src must be the pristine package`
Upstream text already mentions both skip codes	NOT_NEEDED: `extraction/index.js already records data_fragment_skipped / binary_content_skipped`
`const MAX_FILE_SIZE = 1024 * 1024;` altered	**Refuse**: `datafrag1: anchor 'const MAX_FILE_SIZE' occurs 0x in lib/dist/extraction/index.js (expected exactly 1) — upstream shape changed, re-derive the patch`

A pure-digit non-C file or a fragment below 200 KiB is **not** skipped (classifier rules above); such files still go to the parser. A NUL-containing file is skipped regardless of extension or size.

Related scripts

fk_index_patch, siblings fkidx1, lockfix1, resolve1, resolve2, mcpro1, codegraph_safe, owed_report.

Last verified

2026-09-26, scratch copy of stock 1.6.0 `extraction/index.js` only; the real runner cache and every `.codegraph` were untouched.